Unidad 4: Flujos de trabajo colaborativo

Esta semana nos movemos del dominio técnico individual a la **ingeniería de software colaborativa**. Tras entender la arquitectura de Git, el desafío ahora es coordinar el talento humano y el flujo de código para que la velocidad de desarrollo no comprometa la estabilidad del producto.

Comenzaremos analizando las **estrategias de branching**, donde aprenderás a evaluar y seleccionar el modelo de ramificación —ya sea el estructurado **GitFlow**, el ágil **GitHub Flow**, o el dinámico **Trunk-based Development**— que mejor se adapte al tamaño de tu equipo y a la cadencia de entrega requerida. Luego, profundizaremos en la **integración colaborativa**, profesionalizando el uso de **Pull Requests** y **Code Reviews** como herramientas fundamentales no solo para el control de calidad, sino como espacios clave para la transferencia de conocimiento técnico. Finalmente, abordaremos la **estandarización del repositorio**, aprendiendo a configurar entornos profesionales mediante guías de contribución, plantillas de issues y la implementación de **ADRs (Architecture Decision Records)**, asegurando que la documentación y las decisiones de diseño evolucionen a la par del código.

1. ¿Qué son las Estrategias de Branching?

Una estrategia de branching (o modelo de ramificación) es un **conjunto de reglas y convenciones** que definen cuándo se crean las ramas, cómo se nombran, con qué frecuencia se integran y hacia dónde deben fluir los cambios.

Sin una estrategia, el grafo de commits se convierte en una "madeja" ininteligible de ramas cruzadas. El objetivo de estas estrategias es:

- **Aislar el trabajo:** Permitir que los desarrolladores trabajen en nuevas funcionalidades sin romper la versión que está en producción.
- **Facilitar la Integración Continua (CI):** Definir puntos claros donde el código debe ser probado y validado.



- **Estabilizar versiones:** Controlar qué código está listo para ser desplegado y cuál sigue en desarrollo.

2. Modelos de Ramificación Principales

2.1. GitFlow: El Estándar Estructurado

Es el modelo más robusto, diseñado para proyectos con **ciclos de lanzamiento programados** (ej. una actualización cada 15 días). Se basa en la existencia de dos ramas principales de larga duración:

- **`master` / `main`:** Solo contiene código en producción. Cada commit aquí es una versión estable y suele llevar una **Tag**.
- **`develop`:** La rama de integración. Aquí se unen todas las funcionalidades que están listas para la próxima versión.
- **Ramas de apoyo (Temporales):**
  - `feature/`: Para desarrollar nuevas funcionalidades. Nacen de `develop` y vuelven a `develop`.
  - `release/`: Para preparar una nueva versión (ajustes finales y corrección de bugs).
  - `hotfix/`: Para arreglos urgentes que deben ir directo a producción sin esperar a `develop`.

2.2. GitHub Flow: Agilidad y Despliegue Continuo

Es una estrategia simplificada, ideal para la web y entornos donde se despliega **muchas veces al día**.

- Solo hay una rama principal: **`main`**. Todo lo que está en `main` es desplegable.
- Para cualquier cambio, se crea una rama descriptiva (ej: `add-login-button`).



- Se realizan commits localmente y se abre un **Pull Request** para discutir y revisar el código.
- Una vez aprobado y pasado el test, se mergea a `main` y se despliega inmediatamente.

2.3. GitLab Flow: Integración con Entornos

Es una variante que añade ramas para gestionar distintos **entornos de despliegue** (Staging, Pre-producción, Producción).

- El código fluye de forma unidireccional: **`main`** → **`production`**.
- Evita la complejidad de GitFlow pero permite tener una separación clara si tu empresa no puede desplegar a producción de forma instantánea (por ejemplo, si requiere una validación manual del cliente en un servidor de pruebas).

2.4. Trunk-based Development: El Modelo de Alto Rendimiento

Es el preferido por equipos senior y empresas de escala masiva (como Google o Facebook).

- Los desarrolladores integran sus cambios en la rama principal (`trunk` o `main`) **al menos una vez al día**.
- No hay ramas de larga duración; las ramas de funcionalidades duran apenas unas horas.
- **El desafío:** Requiere una batería de tests automáticos extremadamente sólida y el uso de **Feature Flags** (permiten subir código "apagado" que se activa luego por configuración).

3. Evaluación y Selección del Modelo

Seleccionar una estrategia de ramificación no es una decisión meramente técnica, sino una **decisión arquitectónica y cultural** que impacta directamente en la operatividad del equipo. La elección correcta depende de encontrar el equilibrio entre la libertad del desarrollador y el control de calidad necesario para el



negocio. Para evaluar qué modelo implementar, es fundamental analizar cuatro dimensiones críticas: la estructura de comunicación del equipo, la velocidad de entrega requerida por el mercado, el costo operativo de integrar los cambios y, fundamentalmente, la madurez técnica y de automatización disponible en la organización.

3.1. Tamaño y Estructura del Equipo

El factor clave aquí no es solo cuántas personas hay, sino cómo se comunican.

- **GitFlow (Jerárquico):** Es ideal para organizaciones con estructuras de aprobación rígidas (bancos, software médico o gubernamental). Al tener tantas ramas intermedias, permite que un "Release Manager" controle exactamente qué entra en cada versión. Es un modelo defensivo.
- **GitHub Flow (Colaborativo):** Funciona mejor cuando el equipo confía en la revisión por pares (Code Review). Es democrático: cualquier desarrollador puede proponer un cambio a la rama principal directamente.
- **Trunk-based (Sincronizado):** Requiere una alineación total. Si un desarrollador sube código roto, rompe el trabajo de todos los demás *al instante*. Por eso se reserva para equipos de alto rendimiento donde la comunicación es constante y los desarrolladores son responsables de la calidad de cada línea que suben.

3.2. Frecuencia de Entrega (Time-to-Market)

Aquí la pregunta es: **¿Qué tan rápido necesitas que una idea llegue al usuario?**

- **GitFlow (El tren de carga):** El código se acumula en la rama `develop` y "sale" en fechas programadas. Es lento, pero da una sensación de seguridad manual.

- **GitHub Flow (La mensajería):** Los cambios fluyen a medida que se terminan. Es ideal para servicios web donde puedes corregir un error en minutos si algo sale mal tras el despliegue.
- **Trunk-based (El flujo de agua):** Es la meta del **Continuous Deployment**. El código está en constante movimiento hacia producción. Las empresas que hacen miles de despliegues al día (como Netflix o Amazon) usan este modelo porque no pueden permitirse esperar a que una rama de "Release" se cierre.

3.3. Complejidad de Fusión (Merge Hell)

Este es el costo oculto de cada estrategia.

- **GitFlow (Alta):** El problema aquí es el "aislamiento prolongado". Si una rama de funcionalidad (`feature`) vive durante 3 semanas, cuando intentes integrarla, el código base habrá cambiado tanto que los conflictos de merge serán masivos y dolorosos.
- **Trunk-based (Mínima):** Al integrar cambios pequeños varias veces al día, los conflictos son minúsculos y fáciles de resolver. El costo es que debes aprender a trabajar en "pedazos pequeños" que no rompan el sistema.

3.4. Requisitos Técnicos y Cultura

Aquí es donde la mayoría de los equipos fallan al intentar cambiar de modelo.

- **GitFlow (Disciplina):** No requiere grandes herramientas de automatización, pero requiere que todos respeten las reglas de: "no mergear a master sin pasar por release".
- **GitHub Flow (Cultura de PRs):** El cuello de botella aquí son las personas. Si los desarrolladores tardan 2 días en revisar un Pull Request, este modelo fracasa porque detiene la velocidad del equipo.
- **Trunk-based (Automatización Extrema): No se puede hacer Trunk-based sin Tests Automáticos.** Como no hay una fase de "prueba manual"

en una rama de release, la única red de seguridad son los tests que corren en el pipeline de CI. Si no hay tests, el repositorio colapsará en horas.

3.5. Otras Consideraciones Críticas

A. El uso de Feature Flags (Interruptores de Código)

En **Trunk-based**, ¿cómo subes una funcionalidad que te toma 10 días completar si debes integrar todos los días?

- **La solución:** Subes el código "apagado" tras un condicional (`if feature_active: ...`). El código llega a producción, pero el usuario no lo ve hasta que tú activas el interruptor. Esto permite separar el **Despliegue** (subir el código) del **Lanzamiento** (activar la función).

B. El costo de la "Deuda de Integración"

GitFlow acumula deuda. Cuanto más tiempo pase una funcionalidad en su propia rama, más cara es de integrar. Trunk-based paga esa deuda diariamente en cuotas pequeñas.

C. Soporte a Versiones Antiguas

Si vendes un software que los clientes instalan y debes dar soporte a la "Versión 1.2" mientras desarrollas la "2.0", **GitFlow** o **GitLab Flow** son casi obligatorios. GitHub Flow y Trunk-based asumen que solo existe **una versión viva**: la que está corriendo ahora en la nube.

3.6. Tabla comparativa

Para facilitar la selección del modelo adecuado, la siguiente tabla comparativa sintetiza las diferencias fundamentales entre las estrategias más utilizadas en la industria. Esta matriz permite visualizar de forma rápida cómo cada flujo de trabajo escala según el perfil del equipo y cuáles son los requisitos mínimos (tanto humanos como técnicos) para asegurar que la integración de código sea un proceso fluido y no un cuello de botella para la entrega de valor.



Factor	GitFlow	GitHub Flow	Trunk-based
Tamaño del Equipo	Equipos grandes / Jerárquicos	Equipos pequeños / Medianos	Equipos Senior / Ágiles
Frecuencia de Entrega	Semanal / Mensual	Diaria	Continua (múltiples veces al día)
Complejidad de Fusión	Alta (muchos conflictos al final)	Baja	Mínima (se integra constante)
Requisito Técnico	Disciplina de ramas	Cultura de PRs	Tests automáticos obligatorios

4. Procesos de Integración Colaborativa

La integración no es simplemente ejecutar un `merge`. En un entorno profesional, la integración es un **proceso de control de calidad y transferencia de conocimiento** que ocurre principalmente en la plataforma de hosting (GitHub, GitLab, Bitbucket).

4.1 Pull Requests (PR) / Merge Requests (MR)

Un Pull Request no es un comando de Git, sino una **petición formal** para integrar los cambios de una rama (origen) en otra (destino, generalmente `main` o `develop`).

Anatomía de un PR Profesional:

- **Título claro:** Debe resumir el cambio (ej: `feat: agregar validación de RUT en el registro`).
- **Descripción detallada:** Explicar el *qué*, el *cómo* y el *por qué* del cambio.
- **Checklist de validación:** Confirmar que se corrieron los tests, que no hay deuda técnica y que se actualizó la documentación.



- **Evidencia:** Capturas de pantalla o logs que demuestren que el cambio funciona.

4.2 Code Reviews: El Filtro de Calidad

La revisión de código es una conversación técnica. Su objetivo no es "encontrar culpables", sino asegurar que el código sea mantenible, seguro y siga los estándares del equipo.

Roles en la Revisión:

1. **Autor:** Responsable de explicar su código y responder a los comentarios de forma constructiva.
2. **Revisores (Maintainers/Peers):** Encargados de leer el código buscando errores de lógica, problemas de arquitectura o falta de tests.

¿Qué buscar en una Code Review?

- **Legibilidad:** ¿Se entiende el nombre de las variables? ¿Es demasiado complejo?
- **Principios SOLID / DRY:** ¿Estamos duplicando lógica? ¿Las funciones tienen una sola responsabilidad?
- **Seguridad:** ¿Hay posibles inyecciones de código o exposición de datos sensibles?
- **Performance:** ¿Hay bucles innecesarios o consultas pesadas a la base de datos?

4.3 Estrategias de Integración (Merge vs. Squash vs. Rebase)

Al aprobar un PR, el equipo debe decidir cómo se integrará ese trabajo en la rama principal. Esta decisión define la "personalidad" del repositorio: ¿preferimos un registro histórico exacto o una línea de tiempo fácil de leer?



- **Create a Merge Commit (Historial Verídico):** Mantiene todos y cada uno de los commits realizados en la rama de funcionalidad, unidos por un "commit de merge".
    - **Filosofía:** Prioriza la **trazabilidad total**. Permite ver exactamente cuándo se bifurcó el trabajo y qué pruebas se hicieron, aunque el grafo de Git pueda volverse complejo o "sucio" con commits menores (ej: "fix typo").
- **Squash and Merge (Historial Limpio):** Comprime todos los commits de la rama de funcionalidad en **un solo commit** antes de unirlo a `main`.
    - **Filosofía:** Prioriza la **legibilidad de la rama principal**. Es la estrategia estándar en proyectos de código abierto y equipos ágiles, ya que cada entrada en `main` representa una funcionalidad terminada y probada, eliminando el "ruido" del desarrollo diario.
- **Rebase and Merge (Historial Lineal):** Mueve (o "vuelve a basar") los commits de la rama de funcionalidad al frente de `main` sin crear un commit de merge.
    - **Filosofía:** Prioriza una **progresión lineal perfecta**. El historial parece una línea recta donde las funcionalidades se desarrollaron una tras otra. Requiere cuidado, ya que reescribe el historial y puede complicar la resolución de conflictos si no se domina la técnica aprendida en la Semana 1.

Criterio de elección: La mayoría de las organizaciones modernas optan por Squash and Merge para mantener un `main` impecable, mientras que equipos que requieren auditorías estrictas suelen preferir Merge Commit para no perder ni un detalle de lo ocurrido.

4.4 El Factor Humano y la IA en la Integración

Hoy en día, la IA puede participar activamente en este proceso:



- **IA como Autor:** Puede redactar la descripción del PR analizando los cambios realizados.
- **IA como Revisor:** Puede detectar errores de sintaxis o de seguridad antes de que un humano abra el PR.
- **Transferencia de Conocimiento:** Las Code Reviews son el mejor momento para que los juniors aprendan de los seniors. Si la IA detecta un patrón de error repetitivo, el equipo puede decidir crear un **ADR** para estandarizar la solución.

5. Estandarización de la Documentación del Proyecto

La estandarización reduce la fricción en el equipo. Define el "cómo trabajamos aquí" de forma explícita, evitando discusiones repetitivas en las *Code Reviews*.

5.1 Guías de Contribución (CONTRIBUTING.md)

Es el manual de bienvenida para cualquier desarrollador. Este archivo define:

- **Configuración del entorno:** Pasos para instalar dependencias y levantar el proyecto localmente.
- **Estilo de código:** Reglas de linter, uso de espacios vs. tabs, y convenciones de nombres.
- **Flujo de Git:** Qué rama usar como base y cómo nombrar las ramas de funcionalidades (ej: `feature/ticket-123`).
- **Proceso de PR:** Requisitos mínimos para que un Pull Request sea aceptado (ej: "debe tener 2 aprobaciones y 80% de cobertura de tests").

5.2 Plantillas de Trabajo (Issue & PR Templates)

Para evitar reportes de errores incompletos o PRs sin contexto, se utilizan plantillas predefinidas en la carpeta `.github/ISSUE_TEMPLATE/` o similares.

- **Plantillas de Issues:** Obligan al usuario a reportar el error con: Pasos para reproducir, Comportamiento esperado vs. actual, y Capturas de pantalla.

- **Plantillas de Pull Request:** Aseguran que el autor siempre explique: ¿Qué problema resuelve?, ¿Cómo se probó?, y si afecta a la base de datos o a variables de entorno.

5.3 Registros de Decisiones Arquitectónicas (ADRs)

Un **ADR (Architecture Decision Record)** es un documento corto que registra una decisión técnica importante, el contexto en que se tomó y las consecuencias de dicha elección.

¿Por qué son vitales? En proyectos largos, es común olvidar por qué se eligió una base de datos NoSQL sobre una Relacional, o por qué se usa una librería específica. El ADR evita que el equipo vuelva a discutir lo mismo meses después o que un nuevo integrante revierta un cambio sin entender el trasfondo.

Estructura de un ADR:

1. **Título:** Decisión numerada (ej: `ADR 005: Uso de Redis para caché`).
2. **Contexto:** ¿Qué problema enfrentábamos?
3. **Decisión:** ¿Qué solución elegimos?
4. **Estado:** Propuesto, Aceptado, Superado (si una nueva decisión lo reemplaza).
5. **Consecuencias:** Lo bueno (rapidez) y lo malo (costo extra de infraestructura).

5.4 Garantía de Estándares: El Modelo de Doble Validación

Para que las guías de contribución no sean solo "deseos", se implementan dos capas de control que automatizan la vigilancia del código:

A. Validación Local (Git Hooks)

Es la primera línea de defensa y ocurre en la máquina del desarrollador antes de que el código llegue al servidor.



- **Linters y Formateadores:** Herramientas como ESLint o Prettier que analizan el código estáticamente mientras escribes.
- **Git Hooks (Husky):** Scripts automáticos que se disparan en eventos específicos de Git. El más común es el `pre-commit`, que impide que generes un commit si el código no pasa las reglas de estilo o si los tests unitarios fallan.

B. Validación Remota (Status Checks y Gatekeeping)

Es la barrera final y ocurre en la nube (GitHub/GitLab) durante el **Pull Request**. A diferencia de los hooks, esta capa no se puede saltar ni desactivar localmente.

- **Git como Disparador (Trigger):** Al subir el código, Git "despierta" al orquestador (GitHub Actions, Jenkins) para que inicie la ejecución.
- **Status Checks (Checks de Estado):** El servidor ejecuta el proyecto en un entorno limpio y aislado. Si el resultado es negativo (test fallido, error de compilación), el botón de *Merge* se bloquea automáticamente.
- **Gatekeeping:** Esta configuración protege la rama principal de errores humanos. En este ecosistema, **Git gestiona el historial mientras que la CI/CD valida la calidad de cada incremento.**